

Unit 5

K-Means Clustering

1. What is K-Means Clustering?

K-Means is an **unsupervised learning algorithm** used for **clustering** data into **KKK** distinct groups based on feature similarity.

- **Goal:** Partition data into K clusters where each point belongs to the nearest cluster center (**centroid**).
- **Iterative algorithm** that minimizes the variance within clusters.

2. How K-Means Works?

Step 1: Choose the Number of Clusters K

- The user specifies the number of clusters K .

Step 2: Initialize Cluster Centroids

- Randomly place K centroids in the feature space.

Step 3: Assign Each Data Point to the Nearest Centroid

- Compute **Euclidean distance** between each data point and all centroids.
- Assign the point to the cluster with the nearest centroid.

Step 4: Update Centroids

- Compute the **mean of all points** in each cluster.
- Move the centroid to this new mean position.

Step 5: Repeat Until Convergence

- Reassign points and update centroids iteratively **until centroids stop moving**.

3. Mathematical Formulation

- Objective: Minimize the **within-cluster sum of squares (WCSS)**:

$$J = \sum_{i=1}^K \sum_{x_j \in C_i} \|x_j - \mu_i\|^2$$

where:

- x_j = Data point.
- μ_i = Centroid of cluster i .
- $\|x_j - \mu_i\|^2$ = Squared distance between the data point and the centroid.

4. Example: Customer Segmentation

- ◆ **Dataset:** Age, income, spending habits.
- ◆ **K-Means Steps:**
 - Choose $K = 3$ (three customer groups).

- Assign customers to **clusters based on similarity**.
- Update centroids until convergence.

5. How to Choose KKK? (Elbow Method)

- Plot **WCSS vs. K** and find the **elbow point** where WCSS stops decreasing significantly.
- The ideal KKK is at the "elbow."

6. Advantages of K-Means

- **Simple and Fast** – Works well for large datasets.
- **Scales Well** – Efficient for high-dimensional data.
- **Works with Various Applications** – Image segmentation, anomaly detection.

7. Limitations of K-Means

- **Sensitive to Initial Centroids** – May converge to **local minima**.
- **Requires Predefined K** – Needs Elbow Method or Silhouette Score.
- **Struggles with Non-Spherical Clusters** – Works best with circular clusters.
- ✓ **Solution:** Use **K-Means++** for better centroid initialization.

8. Applications of K-Means

- **Customer Segmentation** – Grouping users based on behavior.
- **Anomaly Detection** – Identifying fraud in transactions.
- **Image Compression** – Reducing color complexity in images.

Adaptive Hierarchical Clustering

1. What is Adaptive Hierarchical Clustering?

Adaptive Hierarchical Clustering is an extension of **Hierarchical Clustering**, where the number of clusters and the hierarchy structure **adapt dynamically** based on data characteristics.

- ✓ **Key Idea:** Unlike standard hierarchical clustering, adaptive clustering adjusts its merging/splitting process **based on data density, distance, or similarity thresholds**.

2. How Hierarchical Clustering Works?

Hierarchical clustering builds a **tree-like structure (dendrogram)** that represents how data points are merged into clusters.

◆ **Two Main Types:**

1. **Agglomerative (Bottom-Up Approach)** – Starts with individual points as clusters and **merges** them step by step.

2. **Divisive (Top-Down Approach)** – Starts with all points in one cluster and **splits** them iteratively.
- ✓ **Adaptive hierarchical clustering enhances this process by dynamically adjusting merging/splitting based on criteria like:**
 - Density-based clustering adjustments.
 - Similarity thresholds that change with data distribution.

3. Adaptive Hierarchical Clustering Algorithm

1. **Compute Distance Matrix:** Measure similarity between data points (e.g., Euclidean, cosine similarity).
2. **Initialize Clusters:** Start with each point as its own cluster.
3. **Adaptively Merge or Split Clusters:**
 - Merge clusters **only if similarity exceeds a dynamic threshold**.
 - Allow **splitting if clusters become too dense or diverse**.
4. **Construct a Dendrogram:** The hierarchy adapts as clusters evolve.
5. **Determine Final Clusters:** Cut the dendrogram at an adaptive level to get meaningful clusters.

4. Key Features of Adaptive Hierarchical Clustering

- **Dynamic Thresholding:** Adjusts distance thresholds based on data distribution.
- **Density Awareness:** Adapts to varying densities in data.
- **Automatic Cluster Detection:** No need to predefine the number of clusters.

5. Example: Adaptive Clustering for Image Segmentation

- ◆ **Dataset:** Pixels in an image.
- ◆ **Steps:**
 - Clusters form **based on pixel similarity** (color, texture).
 - **Merges/splits adjust dynamically** based on image complexity.
 - Final clusters represent **different regions** in the image.

6. Advantages of Adaptive Hierarchical Clustering

- **No Need to Predefine KKK** – Automatically finds the optimal number of clusters.
- **Works Well with Complex Data Structures** – Handles varying densities better than K-Means.
- **Interpretable Results** – Dendrograms help visualize hierarchical relationships.

7. Limitations of Adaptive Hierarchical Clustering

- **Computationally Expensive** – Requires recalculating distances dynamically.
- **Sensitive to Distance Metric Choice** – Poor metric selection can lead to incorrect merges/splits.
- ✓ **Solution:** Use **efficient linkage methods** (e.g., Ward's method) and optimize distance computations.

8. Applications of Adaptive Hierarchical Clustering

- **Genomic Data Analysis** – Clustering DNA sequences based on similarity.
- **Customer Segmentation** – Adapting clusters based on evolving user behavior.
- **Anomaly Detection** – Detecting fraud patterns with adaptive merging.

Gaussian Mixture Model (GMM)

1. What is a Gaussian Mixture Model (GMM)?

A **Gaussian Mixture Model (GMM)** is a **probabilistic clustering algorithm** that assumes data is generated from a mixture of **multiple Gaussian distributions**. Each cluster in GMM is represented by a **multivariate normal (Gaussian) distribution**.

- ✓ **Key Idea:** Unlike K-Means, which assigns each data point to a single cluster, **GMM assigns probabilities to each point belonging to multiple clusters**.

2. How GMM Works?

1. Assume Data is Generated from K Gaussian Distributions

- Each cluster is modeled as a **multivariate normal distribution** with:
 - Mean μ_k (center of the cluster).
 - Covariance Σ_k (shape and orientation).
 - Mixing coefficient π_k (weight of the cluster).

2. Estimate Parameters Using Expectation-Maximization (EM)

- **Step 1: Expectation (E-Step):** Compute probability that each data point belongs to a Gaussian.
- **Step 2: Maximization (M-Step):** Update the parameters (μ_k, Σ_k, π_k) to maximize likelihood.
- **Repeat Until Convergence:** The process iterates until clusters are stable.

3. Assign Probabilities to Each Data Point

- Each data point has a **soft assignment** to clusters (instead of a hard assignment like K-Means).

3. Mathematical Representation

A GMM represents the probability of a data point x as:

$$P(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x|\mu_k, \Sigma_k)$$

where:

- π_k = Mixing coefficient (weight of each Gaussian).
- $\mathcal{N}(x|\mu_k, \Sigma_k)$ = Gaussian probability density function (PDF).

Each Gaussian component is defined as:

$$\mathcal{N}(x|\mu, \Sigma) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x - \mu)^T \Sigma^{-1}(x - \mu)\right)$$

4. GMM vs. K-Means

Feature	GMM	K-Means
Cluster Assignment	Soft (probabilistic)	Hard (fixed)
Cluster Shape	Elliptical (full covariance)	Spherical (equal variance)
Handles Overlapping Clusters	✓ Yes	✗ No
Uses Probability Distribution?	✓ Yes	✗ No

- ✓ **GMM is more flexible than K-Means for complex data distributions.**

5. Example: Clustering Customer Segments

- ◆ **Dataset:** Customer spending habits.
- ◆ **Steps:**
 - Assume customers belong to **3 Gaussian-distributed groups**.
 - Use **EM algorithm** to estimate cluster parameters.
 - Assign **probability scores** for each customer belonging to each group.

6. Advantages of GMM

- **Soft Clustering** – Assigns probabilities instead of rigid labels.
- **Captures Elliptical Clusters** – Unlike K-Means, it models variance.
- **Handles Overlapping Clusters** – Ideal when clusters are not well-separated.

7. Limitations of GMM

- **Sensitive to Initialization** – Poor starting points can lead to bad convergence.
- **Computationally Expensive** – More complex than K-Means due to EM steps.
- **May Converge to Local Optima** – Needs multiple runs for stability.

- ✓ **Solution:** Use K-Means initialization for better starting points.

8. Applications of GMM

- **Speaker Recognition** – Identifying people based on voice features.
- **Image Segmentation** – Separating foreground and background.
- **Anomaly Detection** – Identifying unusual patterns in data.
- **Financial Risk Modeling** – Clustering customers based on spending behavior.

Association Rule Mining and Recommendation Systems

1. Association Rule Mining

Association Rule Mining is a data mining technique used to discover relationships between variables in large datasets. It finds **frequent patterns, correlations, or associations** among items in transactional databases.

- ✓ **Goal:** Identify meaningful rules like "If a customer buys X, they are likely to buy Y."

1.1 Key Terms in Association Rule Mining

- **Itemset:** A collection of items (e.g., {Milk, Bread}).
- **Support (S):** The proportion of transactions that contain an itemset.

$$S(X) = \frac{\text{Transactions containing } X}{\text{Total transactions}}$$

- **Confidence (C):** The probability that when item X is bought, item Y is also bought.

$$C(X \Rightarrow Y) = \frac{S(X \cup Y)}{S(X)}$$

- **Lift (L):** Measures the strength of the association.

$$L(X \Rightarrow Y) = \frac{C(X \Rightarrow Y)}{S(Y)}$$

- $L > 1 \rightarrow$ Strong association.
- $L = 1 \rightarrow$ No association.
- $L < 1 \rightarrow$ Negative association.

1.2 Popular Algorithms for Association Rule Mining

- ✓ **Apriori Algorithm**
 - Iteratively finds frequent itemsets and prunes infrequent ones.
 - Uses a "bottom-up" approach with support thresholds.
- ✓ **FP-Growth Algorithm**
 - Faster than Apriori, builds a **Frequent Pattern Tree (FP-Tree)**.
 - Efficient for large datasets.

1.3 Example: Market Basket Analysis

- ◆ **Dataset:** A supermarket's transaction records.
- ◆ **Discovered Rule:**

$$\{Milk, Bread\} \Rightarrow \{Butter\}$$

- **Support:** 30% (30% of transactions contain {Milk, Bread, Butter}).
- **Confidence:** 80% (80% of {Milk, Bread} purchases include Butter).
- **Lift:** 1.5 (Buying {Milk, Bread} increases the chance of buying Butter by 50%).
- ✓ **Used for cross-selling recommendations.**

2. Recommendation Systems

Recommendation Systems suggest products, content, or services to users based on preferences and behavior.

- ✓ **Goal:** Predict user interests and personalize recommendations.

2.1 Types of Recommendation Systems

Type	Description	Example
Content-Based Filtering	Recommends items similar to what the user likes.	Netflix suggesting movies similar to previously watched ones.
Collaborative Filtering	Recommends based on user-item interactions.	Amazon suggesting products based on what similar users bought.
Hybrid Models	Combines content-based and collaborative filtering.	Spotify suggesting songs using both user preferences and other users' behaviors.

2.2 Collaborative Filtering Approaches

- ✓ **User-Based Collaborative Filtering**
 - Finds users with similar preferences.
 - Example: "People who liked this book also liked..."
- ✓ **Item-Based Collaborative Filtering**
 - Finds items frequently purchased or interacted with together.
 - Example: "Customers who bought this also bought..."
- ✓ **Matrix Factorization (e.g., SVD, ALS)**
 - Factorizes user-item interaction matrices to detect hidden patterns.
 - Used in Netflix's **movie recommendation system**.
- ✓ **Deep Learning for Recommendations**
 - Uses neural networks for personalized suggestions (e.g., **Google's YouTube algorithm**).

2.3 Association Rule Mining in Recommendation Systems

- **Used for cross-selling recommendations** (e.g., suggesting complementary products).

- Works well in e-commerce, supermarkets, and online retail.

3. Applications of Association Rule Mining & Recommendation Systems

- **E-commerce (Amazon, Flipkart)** – Product recommendations.
- **Streaming Services (Netflix, Spotify)** – Personalized content suggestions.
- **Retail (Supermarkets, Walmart)** – Market basket analysis.
- **Healthcare** – Predicting disease associations from patient records.

Association Rules

1. Definition:

Association rules are if-then statements that describe relationships between items in large datasets. They are used in **data mining** to discover interesting patterns, especially in **market basket analysis** and recommendation systems.

- ✓ **Goal:** Identify how items are related and predict future co-occurrences.
- ✓ **Example Rule:**
 - If a customer buys Bread & Butter, they are likely to buy Milk.
 - {Bread, Butter} → {Milk}

2. Key Components of Association Rules

- **Antecedent (X):** The item(s) on the left side of the rule (If X...).
- **Consequent (Y):** The item(s) on the right side of the rule (Then Y...).
- **Support (S):** How often X and Y appear together.

$$S(X \Rightarrow Y) = \frac{\text{Transactions containing } X \text{ and } Y}{\text{Total transactions}}$$

- **Confidence (C):** How often Y appears when X is present.

$$C(X \Rightarrow Y) = \frac{S(X \cup Y)}{S(X)}$$

- **Lift (L):** Measures if X and Y are **truly related or occur by chance**.

$$L(X \Rightarrow Y) = \frac{C(X \Rightarrow Y)}{S(Y)}$$

- **Lift > 1** → Strong association.
- **Lift = 1** → No association.
- **Lift < 1** → Negative association.

3. Example: Market Basket Analysis

Transaction ID	Items Purchased
1	Bread, Milk, Butter

Transaction ID	Items Purchased
2	Bread, Butter
3	Milk, Butter
4	Bread, Milk

Association Rule Example:

$$\{Bread, Butter\} \Rightarrow \{Milk\}$$

- **Support:** $2/4 = 50\%$ (Appears in 2 out of 4 transactions).
- **Confidence:** $2/2 = 100\%$ (100% of {Bread, Butter} buyers also buy Milk).
- **Lift:** $100\%/75\% = 1.33$ (Buying Milk is 1.33 times more likely when Bread & Butter are purchased).

4. Algorithms for Finding Association Rules

- **Apriori Algorithm:** Iteratively finds frequent itemsets and prunes infrequent ones.
- **FP-Growth Algorithm:** Uses a tree structure to mine frequent patterns faster than Apriori.

5. Applications of Association Rules

- **E-commerce (Amazon, Flipkart):** "Customers who bought this also bought..."
- **Retail (Supermarkets, Walmart):** Optimizing product placement.
- **Healthcare:** Identifying symptoms linked to diseases.
- **Fraud Detection:** Finding unusual spending patterns.

Association Rule Parameters

Association Rule Mining extracts relationships between items in large datasets. The strength of these relationships is measured using **three key parameters: Support, Confidence, and Lift**.

1. Support (S)

Definition: The proportion of transactions that contain an itemset. It measures how frequently an itemset appears in the dataset.

Formula:

$$S(X) = \frac{\text{Transactions containing } X}{\text{Total transactions}}$$

Example:

Transaction ID	Items Purchased
1	Milk, Bread, Butter

Transaction ID	Items Purchased
2	Bread, Butter
3	Milk, Butter
4	Bread, Milk

- **Support of {Bread, Butter}**: $S(\{Bread, Butter\}) = \frac{2}{4} = 50\%$
- ✓ **Higher Support = More common itemset**
- ✓ **Used to filter out rare itemsets**

2. Confidence (C)

Definition: The probability that item Y is purchased **given that** item X is purchased. It measures the **reliability** of an association rule.

Formula:

$$C(X \Rightarrow Y) = \frac{S(X \cup Y)}{S(X)}$$

Example:

Rule: {Bread, Butter} $\rightarrow$ {Milk}

- Support of {Bread, Butter, Milk} = $2/4 = 50\%$
- Support of {Bread, Butter} = $2/4 = 50\%$
- **Confidence:** $C(\{Bread, Butter\} \Rightarrow \{Milk\}) = \frac{50\%}{50\%} = 100\%$
- ✓ **Higher Confidence = Stronger rule**
- ✓ **But confidence alone doesn't ensure strong association**

3. Lift (LLL)

Definition: Measures how much more likely item Y is purchased when item X is purchased, compared to random chance.

Formula:

$$L(X \Rightarrow Y) = \frac{C(X \Rightarrow Y)}{S(Y)}$$

Example:

- Support of {Milk} = $3/4 = 75\%$
- Confidence {Bread, Butter} $\rightarrow$ {Milk} = 100%
- **Lift Calculation:**

$$L(\{Bread, Butter\} \Rightarrow \{Milk\}) = \frac{100\%}{75\%} = 1.33$$

- ✓ **Lift > 1 $\rightarrow$ Positive correlation** (strong association).
- ✓ **Lift = 1 $\rightarrow$ No correlation** (random occurrence).

- ✓ **Lift < 1** → **Negative correlation** (unlikely to co-occur).

4. Other Parameters in Association Rule Mining

- **Leverage:** Measures the difference between observed and expected co-occurrence of items.
- **Conviction:** Measures rule strength, considering negative correlations.

5. Summary Table

Parameter	Formula	Meaning
Support (S)	$S(X) = \frac{\text{Transactions containing } X}{\text{Total transactions}}$	Frequency of an itemset
Confidence (C)	$C(X \Rightarrow Y) = \frac{S(X \cup Y)}{S(X)}$	Likelihood of Y given X
Lift (L)	$L(X \Rightarrow Y) = \frac{C(X \Rightarrow Y)}{S(Y)}$	Strength of correlation

6. Applications of Association Rule Parameters

- **E-commerce:** "Customers who bought X also bought Y" (Amazon, Flipkart).
- **Retail:** Market Basket Analysis for store layout.
- **Healthcare:** Identifying disease-symptom correlations.
- **Fraud Detection:** Spotting unusual spending patterns.

Calculating Association Rule Parameters

To calculate **Support, Confidence, and Lift** for an **Association Rule**, follow these steps using a dataset of transactions.

1. Example Dataset

Transaction ID	Items Purchased
1	Milk, Bread, Butter
2	Bread, Butter
3	Milk, Butter
4	Bread, Milk
5	Milk, Bread

We will analyze the rule:

$$\{Bread, Butter\} \Rightarrow \{Milk\}$$

2. Calculating Support

Definition: Support measures how frequently an itemset appears in transactions.

Formula:

$$S(X \Rightarrow Y) = \frac{\text{Transactions containing } X \cup Y}{\text{Total transactions}}$$

Calculation for $\{Bread, Butter\} \rightarrow \{Milk\}$:

- **Transactions containing $\{Bread, Butter, Milk\}$: 2** (Transactions **1 and 5**)
- **Total Transactions: 5**

$$S(\{Bread, Butter\} \Rightarrow \{Milk\}) = \frac{2}{5} = 40\%$$

✓ **Support = 40%**

3. Calculating Confidence

Definition: Confidence measures how often Y appears when X is present.

Formula:

$$C(X \Rightarrow Y) = \frac{S(X \cup Y)}{S(X)}$$

Calculation for $\{Bread, Butter\} \rightarrow \{Milk\}$:

- **Transactions containing $\{Bread, Butter, Milk\}$: 2**
- **Transactions containing $\{Bread, Butter\}$: 2** (Transactions **1 and 2**)

$$C(\{Bread, Butter\} \Rightarrow \{Milk\}) = \frac{2}{2} = 100\%$$

✓ **Confidence = 100%**

4. Calculating Lift

Definition: Lift measures how much more likely Y is purchased when X is purchased compared to random chance.

Formula:

$$L(X \Rightarrow Y) = \frac{C(X \Rightarrow Y)}{S(Y)}$$

Calculation for $\{Bread, Butter\} \rightarrow \{Milk\}$:

- **Support of $\{Milk\}$:**
 - Appears in **4** transactions (Transactions **1, 3, 4, 5**).
 - $S(Milk) = \frac{4}{5} = 80\%$.

$$L(\{Bread, Butter\} \Rightarrow \{Milk\}) = \frac{100\%}{80\%} = 1.25$$

- ✓ **Lift = 1.25 (Moderate Positive Correlation)**

5. Summary of Calculated Parameters

Metric	Value
Support	40%
Confidence	100%
Lift	1.25

6. Interpretation of Results

- **Support (40%)** – The rule applies to 40% of all transactions.
- **Confidence (100%)** – Every time a customer buys Bread & Butter, they also buy Milk.
- **Lift (1.25)** – Customers who buy Bread & Butter are 1.25 times more likely to buy Milk compared to random chance.

7. Applications of These Calculations

- **Retail** – Optimizing store layout by placing related products together.
- **E-commerce** – Amazon recommends "Customers who bought X also bought Y."
- **Healthcare** – Identifying symptom-disease relationships.
- **Fraud Detection** – Spotting unusual spending behaviors.

Recommendation Engines

1. What is a Recommendation Engine?

A **Recommendation Engine** is a system that suggests products, services, or content to users based on their preferences and behavior. It is widely used in **e-commerce, streaming platforms, and personalized marketing**.

- ✓ **Goal:** Predict what a user will like based on past interactions and similar users.

2. Types of Recommendation Systems

Type	Description	Example
Content-Based Filtering	Recommends items similar to those the user has liked before.	Netflix suggesting movies based on past watch history.
Collaborative Filtering	Recommends items based on what similar users liked.	Amazon's "Users who bought this also bought..."

Type	Description	Example
Hybrid Recommendation Systems	Combines content-based and collaborative filtering.	Spotify's song recommendations.

3. Content-Based Filtering

- ✓ Works by analyzing item features and user preferences.
- ✓ Uses **TF-IDF (Term Frequency-Inverse Document Frequency)** or **cosine similarity** to measure similarity between items.

Example: Movie Recommendations

- If a user likes "**Inception**" (Sci-Fi, Thriller), recommend "**Interstellar**" (Sci-Fi, Drama) based on genre similarity.
- **Best for personalized recommendations.**
- **Limitation:** Struggles to recommend **diverse** content (cold-start problem).

4. Collaborative Filtering

- Uses user-item interactions to recommend items.
- Works on the idea that **similar users have similar preferences.**

Types of Collaborative Filtering:

Method	Description
User-Based Collaborative Filtering	Finds users with similar tastes and recommends items they liked.
Item-Based Collaborative Filtering	Finds items frequently bought/watched together and recommends them.
Matrix Factorization (e.g., SVD, ALS)	Decomposes the user-item matrix to discover hidden patterns.

- ✓ **Example: Amazon Recommendations**
- If **User A & B** both bought **Item X**, and **User A** bought **Item Y**, recommend **Item Y** to **User B**.
- ✓ **Good for discovering new content.**
- ✓ **Limitation:** Struggles with **new users (cold-start problem).**

5. Hybrid Recommendation Systems

- **Combines Content-Based and Collaborative Filtering** for better accuracy.
- Example: **Netflix** uses both user behavior (collaborative) and movie features (content-based).

6. Deep Learning for Recommendations

- **Neural Networks (Autoencoders, Transformers)** improve recommendations.
- Used by **Google, Facebook, and YouTube** for personalized content.

7. Applications of Recommendation Engines

- **E-commerce:** Amazon, Flipkart (Product recommendations).
- **Streaming Services:** Netflix, Spotify, YouTube (Movie & Music recommendations).
- **Social Media:** Facebook, Twitter (Friend suggestions).
- **Healthcare:** Drug recommendations based on patient history.

How Recommendation Engines Work

A **Recommendation Engine** works by analyzing user behavior, preferences, and item characteristics to suggest relevant products, content, or services.

- ✓ **Goal:** Predict what a user will like based on past interactions, similar users, or content attributes.

1. Steps in a Recommendation Engine

Step 1: Data Collection

- **Implicit Feedback:** Clicks, views, watch time, purchase history.
- **Explicit Feedback:** Ratings, reviews, likes, and dislikes.

Step 2: Data Preprocessing

- Handle **missing values** (e.g., users who haven't rated items).
- Normalize data (e.g., scale ratings from 1–5 to 0–1).
- Create **user-item interaction matrices**.

Step 3: Choosing a Recommendation Approach

(A) Content-Based Filtering

- Analyzes item features (e.g., genre, keywords, product descriptions).
- Uses **TF-IDF (Term Frequency-Inverse Document Frequency)** or **Cosine Similarity** to measure similarity between items.
- **Example:**
 - If a user likes **"Inception" (Sci-Fi, Thriller)** → Recommend **"Interstellar" (Sci-Fi, Drama)**.

(B) Collaborative Filtering

- Uses **user-item interactions** to recommend items.
- Assumes that **similar users like similar items**.
- **Two types:**
 1. **User-Based Collaborative Filtering:**
 - Finds similar users based on past preferences.

- Example: Amazon's "Users who bought X also bought Y."

2. Item-Based Collaborative Filtering:

- Finds similar items and recommends them.
- Example: Netflix's "If you watched X, you might like Y."

(C) Hybrid Recommendation Systems

- Combines **Content-Based + Collaborative Filtering** for better accuracy.
- **Example:** Netflix recommends based on both **user behavior (collaborative)** and **movie features (content-based)**.

(D) Deep Learning-Based Recommendations

- Uses **Neural Networks (Autoencoders, Transformers)** to improve recommendations.
- Example: YouTube, Facebook, Spotify, TikTok.

4. Evaluation of Recommendation Engines

- ✓ **Metrics Used:**
 - **Precision & Recall:** Measures recommendation relevance.
 - **Mean Squared Error (MSE):** Used in rating prediction models.
 - **Normalized Discounted Cumulative Gain (NDCG):** Evaluates ranking quality.

5. Challenges in Recommendation Systems

- **Cold Start Problem:** Hard to recommend for new users or items.
- **Scalability Issues:** Large datasets require high computational power.
- **Sparsity:** Most users interact with only a small subset of items.
- ✓ **Solutions:**
 - Use **Hybrid Models** to combine content and user data.
 - Implement **Matrix Factorization (e.g., Singular Value Decomposition - SVD)** to reduce sparsity.

6. Applications of Recommendation Engines

- **E-commerce:** Amazon, Flipkart (Product recommendations).
- **Streaming Services:** Netflix, Spotify, YouTube (Movie & Music recommendations).
- **Social Media:** Facebook, Instagram, Twitter (Friend suggestions).
- **Healthcare:** Drug recommendations based on patient history.

Collaborative Filtering

1. What is Collaborative Filtering?

Collaborative Filtering (CF) is a **recommendation system technique** that predicts a user's interests based on the preferences of **similar users** or **similar items**.

- ✓ **Key Idea:** Users with similar behaviors tend to like similar items.

2. Types of Collaborative Filtering

Method	Description	Example
User-Based CF	Finds users with similar tastes and recommends items they liked.	"Users who liked X also liked Y."
Item-Based CF	Finds similar items and recommends them to users who interacted with related items.	"Customers who bought this also bought..."
Matrix Factorization	Decomposes the user-item matrix to uncover hidden patterns.	Netflix's recommendation algorithm (SVD, ALS).

3. How Collaborative Filtering Works?

Step 1: Create a User-Item Interaction Matrix

A matrix where **rows = users** and **columns = items**, filled with ratings or interactions.

User	Movie A	Movie B	Movie C	Movie D
U1	5	?	3	?
U2	4	2	?	5
U3	?	3	4	5

Step 2: Find Similar Users or Items

- **User-Based:** Compare users' preferences using **cosine similarity** or **Pearson correlation**.
- **Item-Based:** Compare item interactions across users.

Step 3: Predict Ratings for Unseen Items

- Use weighted averages of similar users/items.

Step 4: Recommend the Top Items

- Suggest the highest predicted ratings for the user.

4. User-Based Collaborative Filtering

- ✓ **Finds users with similar tastes.**
- ✓ **Steps:**
 1. Compute similarity between users.
 2. Predict missing ratings based on similar users' preferences.
 3. Recommend items with the highest predicted ratings.
- ✓ **Limitation:** Does not work well for **new users (cold start problem)**.

5. Item-Based Collaborative Filtering

- **Finds similar items based on user interactions.**
- **Steps:**

1. Compute similarity between items.
2. Predict user preferences for unseen items based on similar items.
3. Recommend highly similar items.
- **Used in Amazon's "Customers who bought X also bought Y."**
- **Limitation:** Needs a large dataset to find meaningful patterns.

6. Matrix Factorization (SVD, ALS)

- ✓ **Reduces the large user-item matrix into lower dimensions to find hidden patterns.**
- ✓ **Example:**
 - Netflix uses **Singular Value Decomposition (SVD)** to predict movie ratings.
- ✓ **Best for handling sparsity (missing values).**
- ✓ **Limitation:** Computationally expensive for large datasets.

7. Advantages of Collaborative Filtering

- **No need for item descriptions (content-agnostic).**
- **Captures complex relationships between users & items.**
- **Works well for discovering new preferences.**

8. Challenges of Collaborative Filtering

- **Cold Start Problem:** Hard to recommend for new users or new items.
- **Data Sparsity:** Most users interact with only a few items.
- **Scalability Issues:** Large datasets require high computation.
- **Solution:** Use **Hybrid Models** (CF + Content-Based Filtering).

9. Applications of Collaborative Filtering

- **E-commerce (Amazon, Flipkart):** Product recommendations.
- **Streaming Services (Netflix, Spotify, YouTube):** Movie & music suggestions.
- **Social Media (Facebook, Twitter, Instagram):** Friend recommendations.
- **Healthcare:** Drug recommendations based on similar patients.

Content-Based Filtering

1. What is Content-Based Filtering?

Content-Based Filtering (CBF) is a recommendation technique that suggests items similar to those a user has liked before. It analyzes **item features** and matches them to a user's preferences.

- **Key Idea:** If a user likes "**Inception**" (**Sci-Fi, Thriller**), recommend movies with similar genres, actors, or directors.

2. How Content-Based Filtering Works?

Step 1: Represent Items Using Features

- Each item (movie, book, product) is represented by a **feature vector**.
- Example for Movies:
 - **Features:** Genre, director, actors, keywords.

Step 2: Create a User Profile

- A **user profile** is built based on past interactions.
- If a user watches Sci-Fi movies, their profile assigns high importance to Sci-Fi.

Step 3: Compute Similarity

- Recommend items **most similar** to the user's past choices.
- Similarity is measured using:
 - **Cosine Similarity**
 - **TF-IDF (Term Frequency-Inverse Document Frequency)** for text data

Step 4: Recommend Top Similar Items

- Rank items based on similarity score and recommend the best matches.

3. Mathematical Representation

Cosine Similarity Between Two Items (A and B):

$$\text{Similarity}(A, B) = \frac{A \cdot B}{||A|| ||B||}$$

where:

- AAA and BBB are **feature vectors** of two items.
- Higher similarity means items are more related.

4. Example: Movie Recommendation

Movie	Genre	Director	Actors
Inception	Sci-Fi, Thriller	Christopher Nolan	Leonardo DiCaprio
Interstellar	Sci-Fi, Drama	Christopher Nolan	Matthew McConaughey

- If a user likes "**Inception**", recommend "**Interstellar**" because:
 - **Genre Similarity (Sci-Fi)**
 - **Director Similarity (Nolan)**
- **Used in Netflix, IMDb, and movie recommendation platforms.**

5. Advantages of Content-Based Filtering

- **Personalized Recommendations** – Based on individual preferences.

- **No Need for Large User Base** – Unlike Collaborative Filtering, it works even for new users.
- **Transparent Model** – Easy to understand why a recommendation was made.

6. Challenges of Content-Based Filtering

- **Cold Start Problem (New Users)** – Hard to recommend items if no user history exists.
- **Feature Engineering is Required** – Needs structured and meaningful item descriptions.
- **Limited Exploration** – Only recommends similar items, failing to discover diverse interests.
- ✓ **Solution:**
- Use **Hybrid Recommendation Systems** (CBF + Collaborative Filtering).

7. Applications of Content-Based Filtering

- **Streaming Services (Netflix, Spotify, YouTube)** – Recommending movies and music.
- **E-commerce (Amazon, Flipkart)** – Suggesting similar products.
- **News & Article Recommendation (Google News, Flipboard)** – Personalizing news feeds.
- **Healthcare** – Suggesting similar treatments or medications.